

Tutorial Letter A2/0/2024

Machine Learning
COS4852

Year module

School of Computing

IMPORTANT INFORMATION

This document contains the questions for Assignment 2 for COS4852 for 2024.

BAR CODE



CONTENTS

Page

1	INTRODUCTION	3
2	Assignment 2.....	3

1 INTRODUCTION

This document contains a discussion on Assignment 2 for COS4852 for 2024.

2 Assignment 2

Question 1

Read Chapter 4 of Nilsson's book (which you downloaded for Assignment 1). Take special note of section 4.1 and its discussion of decision boundaries and their polarity and the concept of the neural network weight-space. The terms TLU and Perceptron refer to the same construct here (i.e. a neuron with weights and a threshold activation function). Similarly, the terms hyperplane, decision boundary, and decision surface refer to the same concept, which linearly divides a space into two sub-spaces. The space can have any number of dimensions. In a 2-dimensional space the decision boundary is a straight line. There is a direct mapping between the decision boundary(s) and the weights of the neural network.

Go to: <https://www.thomascountz.com/2018/04/13/calculate-decision-boundary-of-perceptron>. This shows you how to draw the decision boundary for a Perceptron using the known weights. We simply use linear algebra of a line.

You can also look at <https://www.thomascountz.com/2018/03/23/perceptrons-in-neural-networks>.

Question 1(a)

Consider the Perceptron in Figure 1. The bias and weights are shown in the figure. It uses the

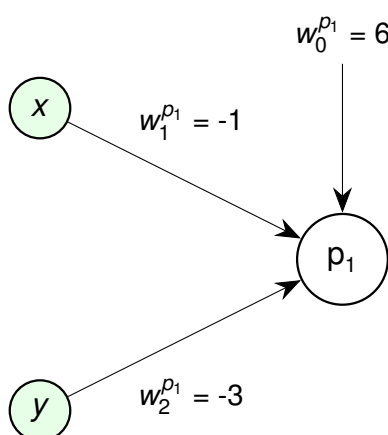


Figure 1: Neuron with weights and bias values.

threshold activation function:

$$f_{threshold}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

The Perceptron classifies the following instances:

$$P_1 = (-4, 4)$$

$$P_2 = (1, 4)$$

$$P_3 = (2, 6)$$

$$P_4 = (6, 2)$$

$$N_1 = (-3, 1)$$

$$N_2 = (1, -5)$$

$$N_3 = (6, -4)$$

$$N_4 = (4, -2)$$

Positive instances are marked as P_i and negative instance as N_i .

- i) Draw the input (x, y) -space for this Perceptron showing the 8 instances.
- ii) Calculate where the decision boundary will be and draw that in the input space. Show all your steps and calculations.
- iii) Does the Perceptron classify the data correctly? If so, prove it. If not, show what can be done to correct this. Show all your steps and calculations (HINT: you do not have to re-calculate the weights).

Question 1(b)

The following 10 instances are marked as positive (P_i) and negative (N_i). A single Perceptron is used to divide the input space into two classes based on the instances. The resulting decision boundary/hyperplane is shown in Figure 2.

$$P_1 = (4, 1)$$

$$P_2 = (2, -1)$$

$$P_3 = (2, 2)$$

$$P_4 = (5, 5)$$

$$P_5 = (3, -2)$$

$$N_1 = (-3, 4)$$

$$N_2 = (-1, 1)$$

$$N_3 = (3, 6)$$

$$N_4 = (-3, -1)$$

$$N_5 = (-2, 3)$$

The cut-off points of the hyperplane are at $(0, 1.25)$ and $(-1, 0)$.

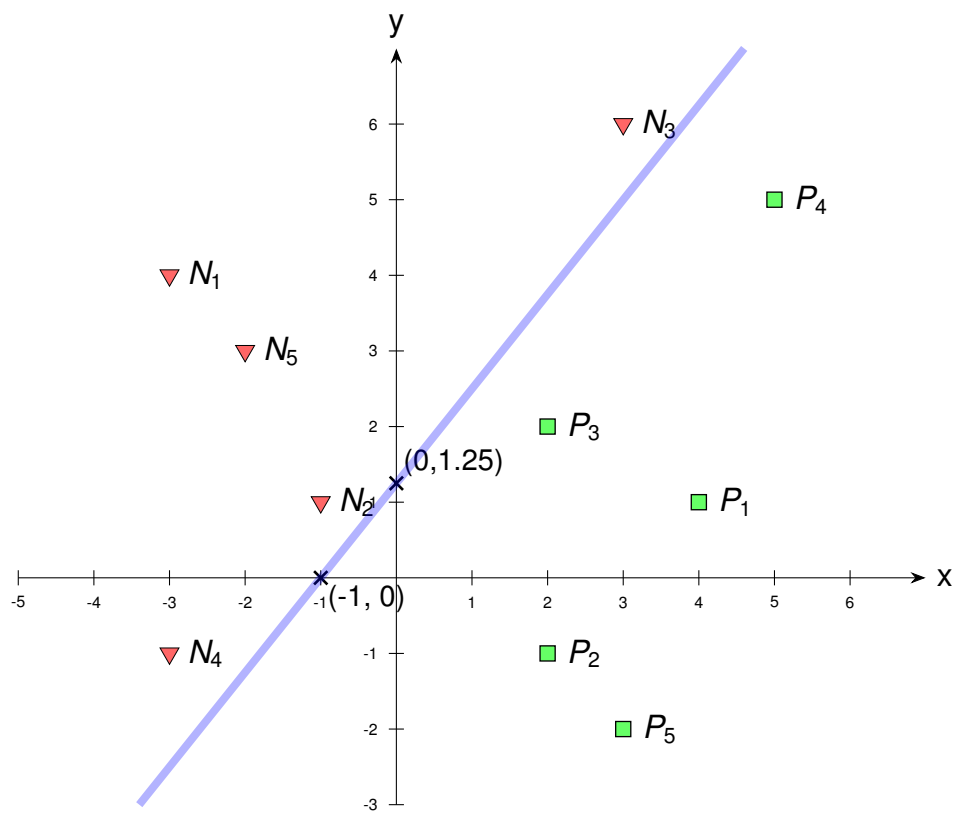


Figure 2: A hyperplane of a Perceptron dividing the input space into positive and negative, using the 10 instances shown.

Calculate the values of the weights and bias of the Perceptron, using the given decision surface/hyperplane.

Question 1(c)

Consider again the same 10 instances.

$$\begin{aligned}
 P_1 &= (4, 1) \\
 P_2 &= (2, -1) \\
 P_3 &= (2, 2) \\
 P_4 &= (5, 5) \\
 P_5 &= (3, -2) \\
 N_1 &= (-3, 4) \\
 N_2 &= (-1, 1) \\
 N_3 &= (3, 6) \\
 N_4 &= (-3, -1) \\
 N_5 &= (-2, 3)
 \end{aligned}$$

A neural network comprising of 2 input neurons (for x and y values), 2 hidden layer neurons (h_1 and h_2), and a single output neuron, was trained to classify the data. The hidden and output neurons use the threshold activation function, which outputs either 0 (for a negative instance) or 1 for a positive instance.

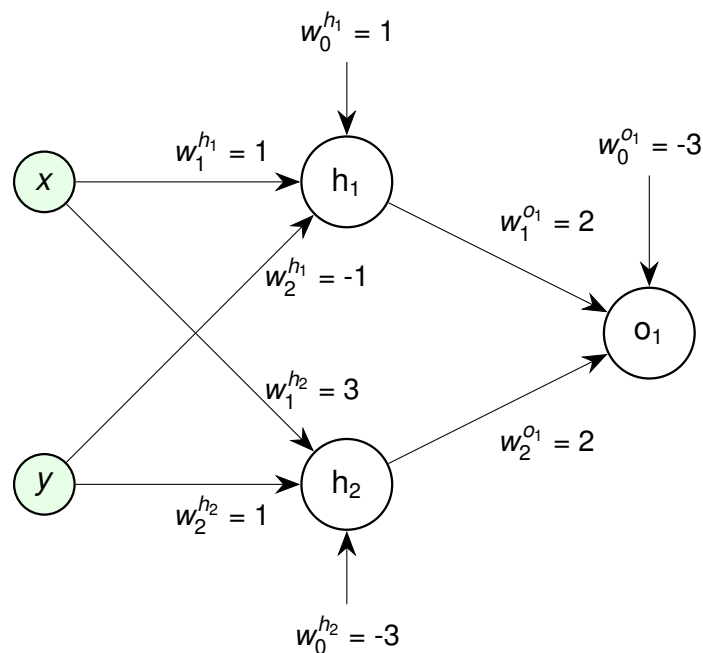


Figure 3: Three-layer neural network with weights and bias values.

- i) Calculate the output for each of the three neurons h_1 , h_2 , and o_1 when N_3 is given as input to the network. Use the weights and bias values as shown in Figure 3.
- ii) Does the network classify N_3 correctly?
- iii) What do you observe about the outputs of h_1 and h_2 when N_3 is given as input? Could the network still correctly classify N_3 ? Explain in detail.
- iv) Draw the decision boundaries for each of the three neurons, h_1 , h_2 , and o_1 . Show all your steps and calculations. Explain what you did to arrive at the diagram for o_1 .

Mark out of 100.

40 or less for clear indication that student does not understand the topic or evidence of plagiarism

50 for a fair understanding

60-70 for understanding and clear well defined examples

80+ for exceptional detail

Question 2

Design neural networks, using one or more Perceptron neurons, for the data as detailed below.

Let your Perceptron use the threshold activation function:

$$f_{\text{threshold}}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

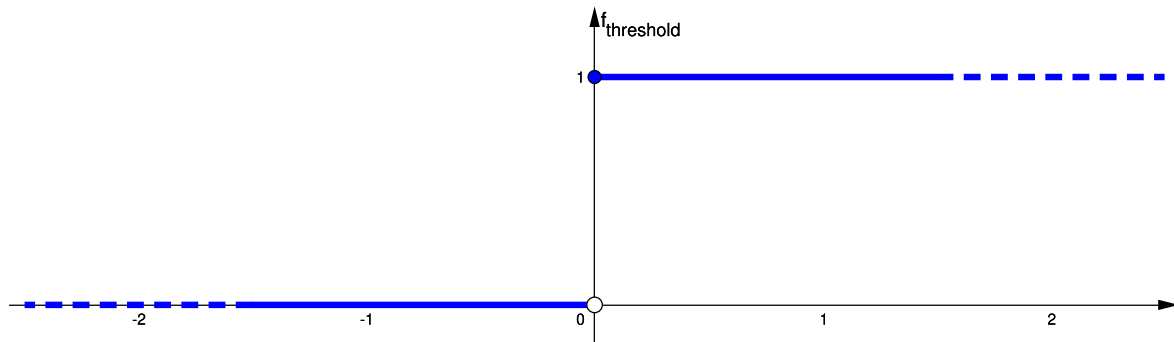


Figure 4: Threshold activation function.

For each sub-question, draw diagrams to show the decision boundaries that your network uses, and the structure of the neural network showing the weights that will correctly classify the function. Show the arguments for your choices, all your assumptions, definitions, and calculations. Prove that the weights you use will correctly classify the function.

Note that this question does **not** ask you to train a neural network using one of the neural network algorithms. The purpose here is similar to the question on the relationship between decision trees and Boolean expressions, in the previous assignment. It is possible to directly map a specific Boolean expression to a network of Perceptrons. Rojas has a very good explanation of the concepts needed in Chapter 2 (and some aspects on XOR in Chapter 6).

Question 2(a)

Data set 1:

$$\begin{aligned} P_1 &= (-3, 3) \\ P_2 &= (3, -3) \\ P_3 &= (3, 3) \\ N_1 &= (-3, -3) \end{aligned}$$

Question 2(b)

Data set 2:

$$P_1 = (-3, -3)$$

$$P_2 = (3, 3)$$

$$N_1 = (-3, 3)$$

$$N_2 = (3, -3)$$

Mark out of 100.

40 or less for clear indication that student does not understand the topic or evidence of plagiarism

50 for a fair understanding

60-70 for understanding and clear well defined examples

80+ for exceptional detail

Question 3

Question 3(a)

Find the original 1986 article by Rumelhart, Hinton, and Williams that introduced the BACKPROPAGATION algorithm, and read it. Give the URL where you found it.

Question 3(b)

Find the following sites and study them:

<https://brilliant.org/wiki/artificial-neural-network/>
<https://brilliant.org/wiki/feedforward-neural-networks/>
<https://brilliant.org/wiki/backpropagation/> <https://www.youtube.com/watch?v=>

The following Python code implements a basic Backpropagation algorithm.

```
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt

# activation functions and their derivatives
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

def relu(x):
    # modify this code
    return x

def relu_derivative(x):
    # modify this code
    return x

def leaky_relu(x, alpha):
    # modify this code
    return x

def leaky_relu_derivative(x, alpha):
    # modify this code
    return x

# activation function selection
activation_function = 'sigmoid'
#activation_function = 'relu'
#activation_function = 'leaky_relu'
```

```

# applying the chosen activation function
def activate(x, func=activation_function):
    if func == 'relu':
        return relu(x)
    elif func == 'leaky_relu':
        return leaky_relu(x, alpha)
    else: # default to sigmoid if not specified
        return sigmoid(x)

def activate_derivative(x, func=activation_function):
    if func == 'relu':
        return relu_derivative(x)
    elif func == 'leaky_relu':
        return leaky_relu_derivative(x, alpha)
    else: # default to sigmoid if not specified
        return sigmoid_derivative(x)

# training data
X = np.array([
    [0, 0, 1],
    [0, 1, 0],
    [0, 1, 1],
    [1, 0, 0],
    [1, 0, 1],
    [1, 1, 0]
])
y = np.array([
    [1],
    [1],
    [0],
    [0],
    [1],
    [0]
])

# initialise weights
np.random.seed(6)
input_size = X.shape[1]
hidden_size = 4
output_size = 1
weights_input_hidden = 2 * np.random.random((input_size, hidden_size)) - 1
weights_hidden_output = 2 * np.random.random((hidden_size, output_size)) - 1

# training
learning_rate = 0.1
alpha = 0.01
epochs = 10000
loss_values = []

for epoch in range(epochs):

```

```

# forward pass
hidden_layer_input = np.dot(X, weights_input_hidden)
hidden_layer_output = activate(hidden_layer_input, activation_function)
final_output = activate(np.dot(hidden_layer_output, weights_hidden_output), activation_function)

# compute error
error = y - final_output
loss = np.mean(np.abs(error))
loss_values.append(loss)

# backward pass
d_predicted_output = error * activate_derivative(final_output, activation_function)
error_hidden_layer = d_predicted_output.dot(weights_hidden_output.T)
d_hidden_layer = error_hidden_layer * activate_derivative(hidden_layer_output, activation_function)

# updating Weights
weights_hidden_output += hidden_layer_output.T.dot(d_predicted_output) * learning_rate
weights_input_hidden += X.T.dot(d_hidden_layer) * learning_rate

if epoch % 1000 == 0:
    loss = np.mean(np.abs(error))
    print(f'Epoch {epoch}, Loss: {loss}')

# final output after training
print(f"Activation function: {activation_function}")
print("Final outputs after training:")
print(final_output)

# Plotting
plt.figure(figsize=(10, 2))
plt.plot(loss_values)
y_max = 0.75
x_max = epochs
plt.xticks(np.arange(0, x_max + 1, 1000))
plt.yticks(np.arange(0, y_max, 0.1))
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.ylim(0, y_max)
plt.xlim(0, epochs)
plt.grid(True)
plt.show()

```

Consider the data in Table 2. This same data is used in the Python code.

Copy and execute the code in a Python 3 interpreter or a Jupyter notebook. Copy the output of your program into your answer. Explain what the output says about the training of the network. Explain the output values in terms of the training data in Table 2.

x_1	x_2	x_3	$f_4(x_1, x_2, x_3)$
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0

Table 1: $f_4(x_1, x_2, x_3)$ **Question 3(c)**

Consider function f_5 :

x_1	x_2	x_3	$f_5(x_1, x_2, x_3)$
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0

Table 2: $f_5(x_1, x_2, x_3)$

Modify the code to set the random seed (HINT: use `np.random.seed(11)` in the right place in the code sets the seed to 11). Use the data from function f_5 , and perform the following experiments:

- (1) Run the code with f_5 .
- (2) Modify the code to use only 2 hidden-layer neurons with f_5 . Try different random seeds.
- (3) Modify the code to use only 1 hidden-layer neuron with f_5 . Try different random seeds.
- (4) Add code to implement the RELU and LEAKY-RELU functions and their derivatives. Keep in mind that these functions uses an α (alpha) parameter that you need to include in the code. Set $\alpha = 0.01$. Run your code with 4 hidden neurons and use f_5 for training. Again, try different random seeds.
- (5) Repeat the last experiment with LEAKY-RELU with various α values.

Show listings of your modifications to the code. Show the outputs from the code. Discuss the results of these experiments, in terms of:

- (i) Whether the training was successful or not.
- (ii) How long it takes to train successfully.
- (iii) What happens to the *Loss* rate.

- (iv) What the output results mean.
- (v) Explain why some were successful and some not.
- (vi) In the experiments where you adjust the α value, explain why the training fails at some point.

Here is a link that explains the most common activation functions used in Backpropagation:
<https://www.baeldung.com/cs/ml-nonlinear-activation-functions>

Question 3(d)

Discuss the different activation functions that can be used in feedforward neural networks, how they work, why they allow training to happen in various algorithms, and their pros and cons. Provide the links of sources you used.

Mark out of 100.

40 or less for clear indication that student does not understand the topic or evidence of plagiarism

50 for a fair understanding

60-70 for understanding and clear well defined examples

80+ for exceptional detail

Question 4

Question 4(a)

Find and read material on Autoencoders in neural networks. Here are some sources to get you started:

<https://www.ibm.com/topics/autoencoder>

<https://www.datacamp.com/tutorial/introduction-to-autoencoders>

<https://www.tensorflow.org/tutorials/generative/autoencoder>

<https://en.wikipedia.org/wiki/Autoencoder>

Copy the following code into a Python 3 interpreter or a Jupyter notebook and execute it. You may need to do this a few times, as the weights are random initial values. From what you've learned about autoencoders, think carefully when the training will be successful.

```
# imports
import torch
import torch.nn as nn
import torch.optim as optim

# define autoencoder model
class Autoencoder(nn.Module):
    def __init__(self):
        super(Autoencoder, self).__init__()
        self.encoder = nn.Linear(8, 3)
        self.decoder = nn.Linear(3, 8)

    def forward(self, x):
        x = torch.sigmoid(self.encoder(x))
        x = torch.sigmoid(self.decoder(x))
        return x

# output tensor in a more readable format
def print_readable_tensor(tensor):
    readable_tensor = torch.round(tensor * 1000) / 1000
    print(readable_tensor)

# data
data = torch.tensor([
    [0,0,0,0,0,0,0,1],
    [0,0,0,0,0,0,1,0],
    [0,0,0,0,0,1,0,0],
    [0,0,0,0,1,0,0,0],
    [0,0,0,1,0,0,0,0],
    [0,0,1,0,0,0,0,0],
    [0,1,0,0,0,0,0,0],
    [1,0,0,0,0,0,0,0]
], dtype=torch.float32)
```

```

# initialise the model, loss function, and optimiser
model = Autoencoder()
criterion = nn.MSELoss()
optimiser = optim.Adam(model.parameters(), lr=0.001)

# training loop
epochs = 20000
for epoch in range(epochs):
    output = model(data)
    loss = criterion(output, data)

    optimiser.zero_grad()
    loss.backward()
    optimiser.step()

    if (epoch+1) % 1000 == 0:
        print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}')

# test the model
with torch.no_grad():
    encoded = torch.sigmoid(model.encoder(data))
    decoded = model(data)
    print('Encoded Representation:')
    print_readable_tensor(encoded)
    print('Decoded Output:')
    print_readable_tensor(decoded)

```

Show the output from the program. What does the progress of the Loss values tell us about what is happening to the training. From what you've learned about autencoders, explain in detail what the two tensors tell us about what the network is doing with encoding and decoding the dataset.

Mark out of 100.

40 or less for clear indication that student does not understand the topic or evidence of plagiarism

50 for a fair understanding

60-70 for understanding and clear well defined examples

80+ for exceptional detail